

PASTEBIN

System Design

High-Level Design | Principal Engineer Reference

Document Generated: March 24, 2026

Table of Contents

1. Requirements & Scale Estimation	3
2. Architecture Overview	4-5
3. Component Deep Dive	6
4. Data Model & Schema Design	7
5. API Design	8
6. Scaling & Caching Strategy	9
7. Failure Modes & Mitigation	10
8. Design Decisions & Trade-offs	11
9. Data Flow Diagrams	12

Requirements & Scale Estimation

Functional Requirements:

- Create a paste with optional expiry, custom alias, and public/private visibility
- Read a paste by ID with efficient caching
- Delete a paste (owner only)
- User accounts with optional authentication
- Paste expiry support (1hr, 1 day, 1 week, 1 month, never)

Non-Functional Requirements:

Availability	99.99% uptime (max ~43 minutes downtime/month)
Read Latency	<50ms p99 (read:write ratio 5:1)
Durability	No data loss; multi-region replication
Paste Size	Up to 10MB per paste, text-only
Consistency	Strong consistency for writes, eventual for reads

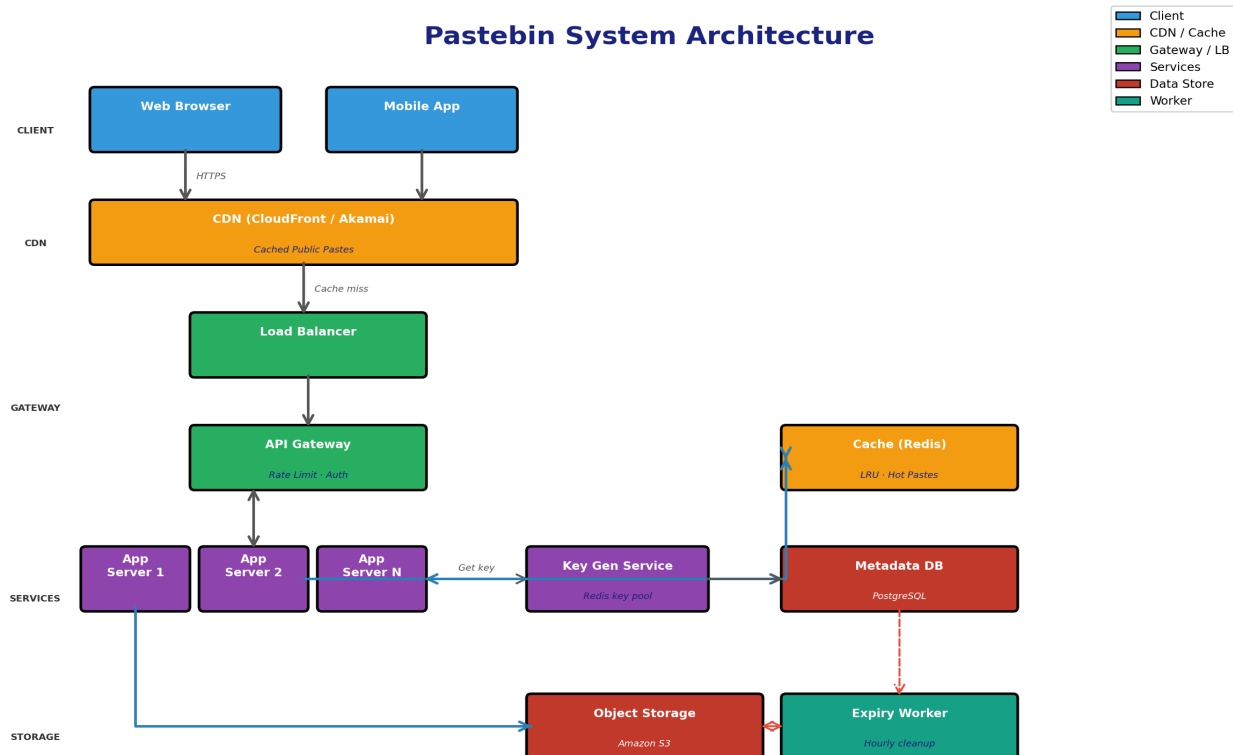
Scale Estimation:

Metric	Value	Notes
Paste Creations/Day	1M	~12 writes/sec
Paste Reads/Day	5M	~58 reads/sec (5:1 ratio)
Average Paste Size	10 KB	
Daily Write Storage	10 GB	1M pastes × 10 KB
Daily Read Bandwidth	50 GB	5M reads × 10 KB
5-Year Storage (no repl.)	18 TB	10 GB/day × 1,825 days
5-Year Storage (3x repl.)	54 TB	With geo-redundancy
Key Space (base62, 6 char)	56.8B keys	More than sufficient

Architecture Overview

The system is organized into distinct layers, each optimized for its responsibility:

- 1. Client Layer:** Web browsers and mobile applications access the system via HTTPS.
- 2. CDN Layer:** CloudFront or Akamai caches public pastes globally, reducing latency and origin load. Cache-Control headers match paste expiry times.
- 3. Load Balancer & API Gateway:** Distributes traffic, validates requests, enforces rate limiting (10 pastes/hr for anonymous, 100/hr for authenticated), and handles authentication tokens.
- 4. Application Servers:** Stateless services process read/write requests. Scale horizontally with auto-scaling groups based on CPU and latency metrics. Each server maintains connection pools to Redis and PostgreSQL.
- 5. Key Generation Service (KGS):** Pre-generates short 6-character base62 keys (aAbBcC...zZ0-9) to avoid collisions and database queries. Maintains a pool of ~1M available keys in Redis. Fallback: on-the-fly UUID generation.
- 6. Object Storage (S3):** Stores paste content as objects with keys like `pastebin-content/{paste_id}.txt`. Provides presigned URLs for direct client downloads. Lifecycle rules move objects to Glacier after 6 months of no access.
- 7. Metadata Database (PostgreSQL):** Stores paste metadata (ID, user_id, created_at, expiry_at, is_public). Uses read replicas for scaling read traffic. Indexes on user_id and expiry_at for common queries.
- 8. Cache Layer (Redis):** LRU cache of hot pastes. Serves ~80% of reads from top ~20% pastes. Cache TTL matches paste expiry. On paste deletion, cache is invalidated immediately.
- 9. Expiry Worker:** Background cron job runs hourly. Lazy deletion: checks expiry on read (return 410 Gone), and actively deletes expired rows from PostgreSQL and S3. Handles cleanup of dangling content.



Key Design Rationale:

- **Content ↔ Metadata Separation:** Different access patterns (sequential reads of content vs. metadata queries). S3 scales independently, metadata DB remains lightweight.
- **Pre-Generated Keys:** Avoid race conditions in database, enable O(1) lookup in Redis, faster response times.
- **Presigned URLs:** Direct S3 downloads bypass app servers, reducing bandwidth and latency for large pastes.

Component Deep Dive

Key Generation Service (KGS)

The KGS maintains a pool of pre-generated base62 keys to ensure fast, collision-free paste creation:

Architecture: Two Redis sets are maintained:

- **keys_available:** Pool of unused keys (target: 1M ahead of write rate)
- **keys_used:** Set of allocated keys for O(1) collision detection

On Paste Creation:

1. App server calls `KGS.GetKey()` → Redis SPOP `keys_available`
2. Validate uniqueness (check `keys_used`)
3. Add to `keys_used` (atomic Redis operation)
4. Return key to app server (typically <5ms latency)

Refill Process: Background job monitors keys_available size. When <20% threshold reached, generates new base62 strings and preloads into Redis. Generates in batches of 10K for efficiency.

Fallback Strategy: If KGS unavailable (Redis down), app server can generate UUID on-the-fly and fallback to longer keys. Exposes /health endpoint for liveness checks.

Object Storage Design (S3)

Paste content is stored as individual S3 objects:

Key Naming: s3://pastebin-content/{paste_id}.txt

Metadata in S3 Object Tags:

- user_id: Owner ID (for access control validation)
- created_at: Timestamp for audit trails
- expiry_at: For S3 lifecycle rules

Presigned URLs: App servers generate presigned POST/GET URLs (valid for 15 min) to allow direct client ↔ S3 transfers. Reduces app server bandwidth and latency, especially for large pastes.

Lifecycle Management:

- Transition to GLACIER_IR after 6 months of no access (infrequent access tier)
- Delete after 7 years (archive retention policy, configurable)

Durability & Redundancy: S3 Standard provides 11 nines (99.999999999%) durability. Cross-region replication enabled to a standby bucket for disaster recovery.

Paste Expiry & Cleanup

Two-pronged approach ensures no zombie pastes:

Lazy Deletion on Read: When client requests GET /pastes/{id}, app checks expiry_at < NOW(). If expired, return 410 Gone and remove from Redis cache.

Active Cleanup Cron: Hourly background job:

1. DELETE FROM pastes WHERE expiry_at < NOW() - INTERVAL '1 hour' (batch delete)
2. List S3 objects with matching tags, initiate delete
3. Log deleted IDs to CloudWatch for monitoring

Cache TTL Synchronization: Redis EXPIRE command set to match paste expiry. Cache eviction triggers immediate invalidation.

Monitoring: Alert if cleanup job takes >30 min (indicates large backlog or DB performance issues).

Data Model & Schema Design

Pastes Table (PostgreSQL)

Column	Type	Constraints	Notes
paste_id	VARCHAR(6)	PRIMARY KEY	Base62 from KGS
user_id	UUID	FK → users.user_id, NULLABLE	NULL for anonymous
content_url	VARCHAR(255)	NOT NULL	S3 path (s3://bucket/paste_id.txt)
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Creation timestamp
expiry_at	TIMESTAMP	NULLABLE	NULL = never expire
is_public	BOOLEAN	NOT NULL, DEFAULT TRUE	Public visibility flag
views	BIGINT	NOT NULL, DEFAULT 0	Approximate counter (not exact)
updated_at	TIMESTAMP	NOT NULL	Last update time

Users Table (PostgreSQL)

Column	Type	Constraints	Notes
user_id	UUID	PRIMARY KEY	auto-generated
email	VARCHAR(255)	UNIQUE, NOT NULL	Login identifier
password_hash	VARCHAR(255)	NOT NULL	bcrypt with salt
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	
updated_at	TIMESTAMP	NOT NULL	

Key Indexes

Composite Index on (user_id, created_at DESC): Accelerates "get user's pastes" queries with pagination.

Index on expiry_at: Used by cleanup cron to efficiently find expired pastes.

Index on is_public: For public pastes listing (if supported).

Partitioning Strategy (Optional): As table grows, consider range partitioning on created_at (monthly or quarterly). Improves query performance and simplifies retention/archival.

API Design

REST API Contracts

POST /v1/pastes — Create a new paste

```
Request Body: { "content": "string (up to 10MB)", "expiry": "enum [1h, 1d, 1w, 1mo, never]",  
"alias": "string (optional, max 32 chars)", "is_public": "boolean (default: true)", "user_token":
```

```
"string (optional JWT, for authenticated users)" } Response (201 Created): { "paste_id": "abc123", "url": "https://pastebin.com/abc123", "expiry_at": "2026-03-23T15:00:00Z", "created_at": "2026-03-23T14:00:00Z" } Error Cases: • 400 Bad Request: Content too large, invalid expiry • 401 Unauthorized: Invalid token • 429 Too Many Requests: Rate limit exceeded
```

GET /v1/pastes/{paste_id} — Retrieve a paste

```
Response (200 OK): { "paste_id": "abc123", "content": "string", "is_public": true, "created_at": "2026-03-23T14:00:00Z", "expiry_at": "2026-03-23T15:00:00Z", "views": 42, "owner": "user_id (if public)" } Error Cases: • 404 Not Found: Paste doesn't exist • 410 Gone: Paste has expired • 403 Forbidden: Private paste, unauthorized access
```

DELETE /v1/pastes/{paste_id} — Delete a paste (owner only)

```
Headers: Authorization: Bearer {user_token} Response (204 No Content): Success Error Cases: • 401 Unauthorized: Missing/invalid token • 403 Forbidden: User is not the owner • 404 Not Found: Paste doesn't exist
```

GET /v1/users/{user_id}/pastes — List user's pastes (paginated)

```
Query Parameters: limit: integer (default: 20, max: 100) cursor: string (opaque pagination token, optional) Headers: Authorization: Bearer {user_token} Response (200 OK): { "pastes": [ {"paste_id": "abc123", "created_at": "...", "is_public": true}, ... ], "next_cursor": "eyJpZCI6ImFiYzEyMyJ9" (base64 opaque token) }
```

Scaling & Caching Strategy

Read Path Optimization

CDN Caching (First Layer): CloudFront/Akamai caches public pastes globally.

- Cache-Control: public, max-age={seconds_until_expiry}
- Cache key includes paste_id and API version
- Geo-distributed POP (point of presence) reduces latency to <30ms for 95% of users

Redis Cache (Second Layer - LRU): In-memory cache of hot pastes.

- Cache key: paste:{paste_id} → full paste JSON (content + metadata)
- Capacity: ~100GB (1M pastes × 100KB avg including metadata)
- Pareto principle: top 20% of pastes account for 80% of reads → cached
- Automatic eviction: LRU policy when capacity exceeded
- TTL matching: Redis EXPIRE set to (expiry_at - now()), so cache entry expires with paste

Database (PostgreSQL): Final authoritative source.

- Read replicas scale horizontal read throughput (e.g., 3 replicas)
- Replication lag: typically <100ms (acceptable for metadata)
- Query optimization: indexes on user_id, expiry_at, is_public

Read Hit Rate Analysis:

- CDN hit rate: ~60% (public pastes)
 - Redis hit rate: ~35% (hot pastes)
 - DB queries: ~5% (cold/private pastes)
- Combined = 95% requests avoid DB hit

Write Path & Database Scaling

Stateless App Servers: Write requests distribute uniformly across app servers.

- Auto-scaling group targets CPU >70% or latency >100ms
- Typical scale: 10–100 servers based on traffic
- Connection pooling: 50 connections per server to DB (500–5000 concurrent connections to DB)

Key Generation Service (KGS) Scaling:

- Single KGS instance backed by Redis cluster: potential bottleneck
- Mitigation: Replicate with primary-replica setup, shard by key prefix if needed (62 shards max)
- Pre-generate keys in batches to amortize latency

Database Write Optimization:

- Single PostgreSQL primary accepts writes (RDS Multi-AZ failover for HA)
- Batch inserts where applicable (not applicable here, single paste per request)
- Write performance: ~1000 inserts/sec on modern hardware (sufficient for 12 writes/sec)
- Sharding strategy (if scaling beyond 1M writes/sec): Shard by first char of paste_id (62 shards)

S3 Write Scalability:

- S3 automatically scales for millions of requests/sec
- For large objects, use multipart upload API (parallel streams)
- No specific tuning required at current scale

Rate Limiting

Implemented at API Gateway using token bucket algorithm:

- **Anonymous Users:** 10 pastes/hour, 100 reads/hour per IP
- **Authenticated Users:** 100 pastes/hour, 1000 reads/hour per user
- Enforcement: Redis-backed counter with sliding window (1-hour windows)
- Response: 429 Too Many Requests with Retry-After header

Failure Modes & Mitigation

Failure Scenario	Impact	Mitigation Strategy
KGS (Redis) Down	Cannot create new pastes	Fallback to UUID generation on-the-fly; app maintains local key cache
S3 Unavailable	Cannot read/write content; reads return errors	Multi-region S3 replication; failover to standby bucket in seconds
Redis Cache Down	Increased DB load, higher latency (50–200 ms)	App gracefully bypasses cache; queries DB directly. K8s auto-restarts
PostgreSQL Primary Down	Write operations fail; reads from replicas only	RDS Multi-AZ automatic failover (~1 min). Use read replicas as fallback
API Gateway/LB Down	All traffic blocked	Multiple LB instances across AZs; health checks every 5 sec
Expiry Worker Down	Expired pastes linger; ghost data accumulates	Lazy deletion on read (return 410). Worker restarts via K8s. Alert if backlog
App Server Crash	Request loss, reduced capacity	Stateless design allows seamless failover. Health checks (TCP:8080)

Recovery Time Objectives (RTO) & Recovery Point Objectives (RPO)

RTO (Time to Restore Service):

- Single component failure: <30 seconds (auto-scaling/health checks)
- Data center failure: <5 minutes (multi-region failover)

RPO (Data Loss Window):

- Database: <1 second (synchronous replication to standby)
- S3: <1 minute (async cross-region replication)
- Cache: Zero tolerance (lossy, reconstructible from DB/S3)

Key Design Decisions & Trade-offs

Decision	Option A	Option B	Chosen	Rationale
Key Generation	Generate UUID on-the-fly	Pre-generated KGS with KGS62	KGS62	Avoids collision races, O(1) lookup, fast response, shorter user URLs
Content Storage	BLOB in PostgreSQL DB	Object Storage (S3)	S3	Cost-effective (\$0.023/GB vs \$0.10+/GB DB), unlimited scale, CDN integration
Consistency Model	Eventual Consistency	Strong Consistency	Strong (Writes)	Data durability critical; eventual OK for reads due to low volume of content
DB Technology	NoSQL (DynamoDB)	Relational (PostgreSQL)	PostgreSQL	Relational ownership (user-paste FK), ACID transactions, complex queries
Cache Strategy	Write-through	Write-around (Lazy)	Write-around	Most pastes read once (no cache hit reuse); avoids cache pollution and invalidation
Expiry Cleanup	Eager (immediate delete)	Lazy (on-read check)	Hybrid	Lazy on read minimizes latency; active job prevents ghost data accumulation

Alternative Approaches Considered

Distributed Key-Value Store (Redis/Memcached for Content): Rejected due to lack of persistence and durability guarantees. S3 provides 11 nines durability at lower cost.

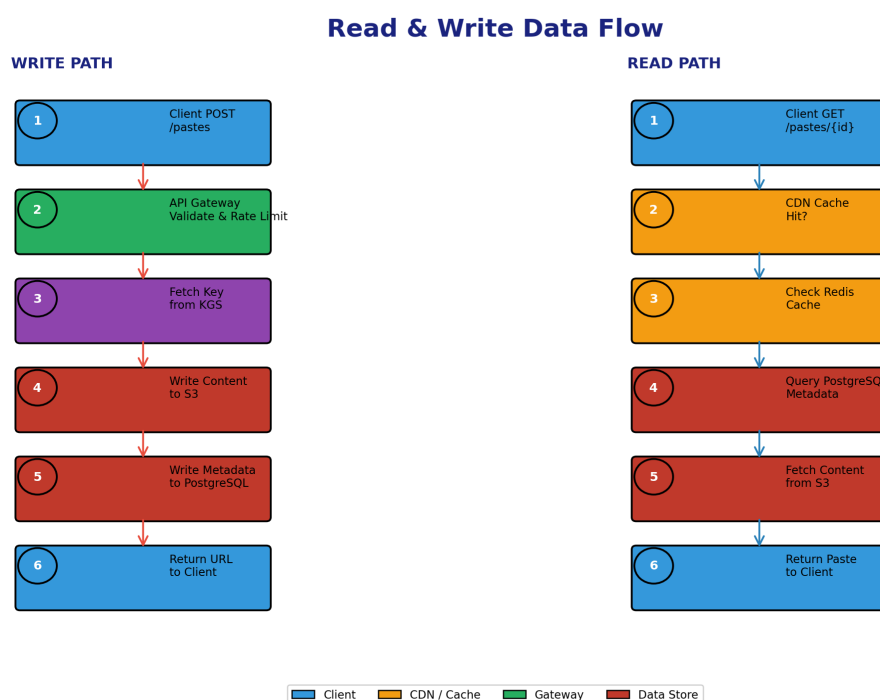
Message Queue (Kafka) for Write Scaling: Added complexity without benefit at current scale. Direct DB writes sufficient for 12 writes/sec.

GraphQL API: Considered but rejected; REST simpler for this use case (few entities, simple relationships).

Serverless (Lambda + DynamoDB): Cold starts and DynamoDB costs exceed traditional stack at write scale. Reserved capacity model more cost-effective.

Data Flow Diagrams

The following diagrams detail the step-by-step flow of requests through the system for both write (paste creation) and read (paste retrieval) operations. Numbers indicate sequence order.



Write Path Explanation (Left Side):

- Step 1: Client sends POST request with paste content, expiry, and optional auth token.
 - Step 2: API Gateway validates rate limits and authentication. Returns 429 if over quota.
 - Step 3: App server fetches a pre-generated base62 key from KGS (Redis), ensuring uniqueness.
 - Step 4: App server uploads content to S3 using presigned URL or direct PutObject API.
 - Step 5: App server writes paste metadata (ID, user_id, created_at, expiry_at, S3_url) to PostgreSQL.
 - Step 6: PostgreSQL returns row ID; app server responds to client with paste URL.
- Response time: ~200–400ms (dominated by S3 and DB latency).

Read Path Explanation (Right Side):

- Step 1: Client sends GET request for a specific paste.
- Step 2: CDN checks if public paste is cached (cache key = paste_id). If hit, return content immediately (<30ms).
- Step 3: If CDN miss, app server checks Redis cache for the paste. If cached, return immediately (<10ms).
- Step 4: If Redis miss, query PostgreSQL for metadata (paste_id, expiry_at, is_public, S3_url).
- Step 5: Fetch actual content from S3 using presigned URL (app bypasses via direct client URL).
- Step 6: Populate Redis cache with paste (LRU, TTL = expiry_at), return to client.

Response time: CDN 30ms, Redis 50ms, DB 100–150ms depending on hit rate.

Performance Targets & Monitoring

SLOs (Service Level Objectives)

Metric	Target	Method
Read Latency (p99)	<50ms	Measured from client; includes network + CDN + app + DB
Read Latency (p50)	<20ms	Most requests hit CDN or Redis
Write Latency (p99)	<500ms	Slower due to S3 + DB writes
Availability	99.99%	Multi-AZ deployment, health checks every 5 sec
Data Durability	99.999999999%	S3 Standard + cross-region replication
Cache Hit Rate (overall)	>95%	CDN + Redis combined

Key Metrics & Monitoring

Application Metrics (CloudWatch / Datadog):

- Request latency by endpoint (p50, p95, p99)
- Error rate by type (4xx, 5xx)
- Cache hit rate (CDN, Redis, DB)
- Request throughput (RPS)
- Database connection pool utilization
- S3 request latency and error rate

Business Metrics:

- Pastes created/deleted per day
- Average paste lifetime
- Public vs. private paste ratio
- Expired paste cleanup lag

Infrastructure Metrics:

- CPU, memory, disk usage on app servers
- Redis memory usage and eviction rate
- PostgreSQL query latency (top slow queries)
- Network bandwidth (ingress/egress)
- Multi-AZ replication lag (<100ms target)

Alerts: Trigger paging for p99 latency >100ms, error rate >0.1%, availability <99.9% over 5min window.

Cost Estimation (Annual)

Component	Annual Cost	Notes
PostgreSQL (RDS)	\$50K	3 × db.r5.2xlarge (1 primary, 2 replicas), Multi-AZ

S3 Storage	\$18K	18 TB × \$1/GB/year (infrequent, with Glacier transition)
S3 Requests	\$5K	~50M reads/month, \$0.10 per 1M
Redis (ElastiCache)	\$30K	300 GB cluster (cache, 3 nodes)
CDN (CloudFront)	\$80K	~100 Tbps monthly at \$0.085/GB
App Servers (EC2)	\$120K	~30 t3.xlarge instances, on-demand + reserved discount
Data Transfer (Cross-Region)	\$25K	5 GB/day × 365 × \$0.20/GB (replication)
Monitoring & Logging	\$10K	CloudWatch, DataDog, X-Ray
Load Balancer	\$25K	ALB, ~50K requests/sec
TOTAL	\$~363K	For 1M pastes/day (annual, rough estimate)

Future Enhancements & Roadmap

- Syntax Highlighting:** Integrate Pygments or highlight.js to detect language and render formatted code pastes.
- Search:** Implement Elasticsearch cluster to index paste content. Enable full-text search across pastes (with privacy considerations for private pastes).
- Paste Sharing & Collaboration:** Allow multiple users to edit a single paste in real-time using WebSockets and operational transformation (like Google Docs).
- Version Control:** Store paste history and allow rollback to previous versions. Implement MVCC or event sourcing.
- API Webhooks:** Notify external systems when pastes are created/deleted (e.g., Slack integration).
- User Profiles & Reputation:** Track user metrics (pastes created, views), implement moderation/reporting.
- Custom Domains:** Allow users to host pastes on their own domains via CNAME.
- GraphQL API:** Parallel GraphQL endpoint for complex queries (e.g., user pastes with filtering).
- Machine Learning:** Detect spam/malicious content using ML models. Recommend related pastes.
- Mobile Native Apps:** iOS/Android apps with offline draft support and push notifications for shared pastes.

Summary & Conclusion

System Overview: The Pastebin system design presented in this document is architected for high availability, low latency, and cost efficiency at scale (1M pastes/day). The architecture separates concerns across independent, horizontally scalable layers: clients, CDN, API gateways, stateless app servers, distributed caches, object storage, and a relational database.

Key Strengths:

- **Scalability:** Stateless app servers and managed services (S3, RDS) scale independently.
- **Performance:** Multi-layered caching (CDN → Redis → DB) achieves sub-50ms p99 latency.
- **Durability:** S3 (11 nines) + cross-region replication + PostgreSQL Multi-AZ ensure data safety.
- **Cost-Efficiency:** S3 for content, PostgreSQL read replicas, and lifecycle rules keep costs under control.
- **Availability:** 99.99% uptime through auto-scaling, health checks, and automatic failover.

Trade-offs Made:

- Pre-generated keys vs. on-the-fly UUIDs: Chose pre-generation for simplicity and speed.
- Content in DB vs. S3: Chose S3 for cost and unlimited scale.
- Strong vs. eventual consistency: Strong for writes, eventual for reads.
- Eager vs. lazy expiry cleanup: Hybrid approach (lazy on read + active job).

Operational Considerations:

- Monitoring and alerting are critical; establish dashboards for latency, error rate, and cache hit rates.
- Database partitioning (by created_at) should be planned before reaching 1B+ pastes.
- Implement feature flags for gradual rollout of syntax highlighting, search, and collaboration features.
- Plan for multi-region expansion (e.g., separate stacks in APAC, EMEA) as user base grows.

Conclusion: This design balances performance, reliability, and cost at the expected scale. The system is built on well-proven technologies (PostgreSQL, Redis, S3) with minimal custom code, ensuring maintainability and operational simplicity. The architecture is designed to scale gracefully to 10–100× the current load with incremental additions (sharding, additional caches, microservices) rather than fundamental rewrites.